DATABASE MANAGEMENT SYSTEMS

CT 652 | BE / BCT | Year III / Part II

Tribhuvan University — Institute of Engineering

CHAPTER 6: FILE STRUCTURE & HASHING

Question Bank with Comprehensive Answers — File Organization, Indices, B+ Tree Insertions, Static & Extendible Hashing

SYLLABUS OVERVIEW — Unit 6

Unit	Topic	Hours	Marks
1	Introduction	3 hrs	4
2	Data Models	7 hrs	12
3	Relational Languages and Relational Model	7 hrs	12
4	Database Constraints and Normalization	6 hrs	12
5	Query Processing and Optimization	4 hrs	8
6	File Structure and Hashing	4 hrs	8
7	Transactions Processing and Concurrency Control	6 hrs	12
8	Crash Recovery	4 hrs	6
9	Advanced Database Concepts	4 hrs	6
Total		**45 hrs**	**80**

Chapter 6 Topics (4 Hours | 8 Marks)

- 6.1 File Organization (Heap, Sequential, Hashed, B+ tree)
- 6.2 Ordered Indices — primary, secondary, clustering
- 6.3 Dense and Sparse Indices
- 6.4 Multilevel Indices
- 6.5 Static Hashing — hash function, overflow handling
- 6.6 Dynamic / Extendible Hashing
- 6.7 B-Tree and B+ Tree Index Structure
- 6.8 B+ Tree Insertion and Deletion

CHAPTER 6 — Questions & Comprehensive Answers

Q: What is file organization? Explain heap, sequential, hashed, and B+ tree file organizations with their advantages and disadvantages.
Asked: ALL years 2070–2082 — asked every exam | Marks: [4–6]

Answer

File Organization

File organization refers to the method used to arrange and store records of a relation on disk. The choice of file organization affects the efficiency of data retrieval, insertion, deletion, and update operations. Different organizations suit different access patterns.

1. Heap File Organization (Unordered)

Records are stored in an unordered fashion. New records are simply appended to the end of the file (or to any free space). There is no ordering or grouping based on attribute values.

- Insertion: O(1) — append to end; very fast
- Search: O(n) — must scan entire file (linear scan); slow
- Deletion: Find record, mark as deleted (leaves gaps); periodic reorganization needed
- Update: Find record (linear scan) then update in place

Aspect	Heap File
Structure	Unordered; records stored as inserted
Insertion	Very fast — O(1) append to end
Search	Slow — full sequential scan O(n)
Deletion	Mark deleted; may leave gaps
Best for	Bulk loading, write-heavy workloads, no frequent queries

2. Sequential File Organization (Ordered)

Records are stored in a sorted order based on a search key (ordering attribute). The file is physically sorted on disk by the search key value.

- Search on ordering key: O(log n) blocks — binary search is possible
- Range queries on ordering key: very efficient — sequential read after first match
- Insertion: Expensive — must find correct position and shift records, or use overflow pages
- Deletion: Find and remove; may leave gaps or use deletion markers
- Periodic reorganization needed to reclaim space and restore order

Aspect	Sequential File
Structure	Records sorted by ordering (search) key

Insertion	Slow — must maintain sort order; overflow chains
Search on key	Fast — binary search O(log n)
Range queries	Efficient — sequential scan after first match
Deletion	Mark deleted or shift; periodic reorg needed
Best for	Read-heavy, range queries, ordered reports

3. Hashed File Organization

Records are distributed across buckets using a hash function applied to the search key. The hash function maps key values to bucket addresses, allowing O(1) average-case access.

- Search on hash key: O(1) average — compute hash, go directly to bucket
- Insertion: O(1) average — hash key, insert into bucket
- No support for range queries — hash destroys ordering
- Overflow buckets needed when a bucket fills up

Aspect	Hashed File
Structure	Buckets; records distributed by hash function
Insertion	O(1) average — direct bucket access
Equality search	O(1) average — hash then access bucket
Range queries	Not supported — no ordering
Deletion	Find bucket, remove record
Best for	Point queries (equality); fast insert/lookup by key

4. B+ Tree File Organization

Records are stored in the leaf nodes of a B+ tree, ordered by the search key. Internal nodes contain only keys and pointers (not records). All leaf nodes are linked, enabling efficient range scans.

- Search: O(log n) — traverse tree from root to leaf
- Range query: O(log n + k) — find first key, then scan leaf chain
- Insertion/Deletion: O(log n) — always keeps tree balanced
- Space utilization: at least 50% full at every level (guaranteed by split/merge rules)

Aspect	B+ Tree File
Structure	Balanced tree; records in leaves; leaves linked
Insertion	O(log n) — always balanced via splits
Search	O(log n) — root-to-leaf traversal
Range queries	O(log n + k) — very efficient with leaf chain
Deletion	O(log n) — may merge nodes
Best for	General purpose; both point queries and range queries

Comparison of File Organizations

Operation	Heap	Sequential	Hashed	B+ Tree
Insert	O(1) ✓✓✓	O(n) / overflow	O(1) ✓✓✓	O(log n) ✓✓
Point Search	O(n)	O(log n) ✓✓	O(1) ✓✓✓	O(log n) ✓✓
Range Search	O(n)	O(log n+k) ✓✓	Not supported ✗	O(log n+k) ✓✓
Delete	O(n)	O(n)	O(1)	O(log n)
Ordering	None	By search key	None	By search key

Q: What is indexing? Explain primary index, secondary index, and clustering index with examples. Differentiate dense and sparse indices.

Asked: 2081 Ashwin; 2080; 2078 Poush; 2074; 2073 Magh; 2072 Ashwin; 2071; 2070 Magh | Marks: [4]

Answer

Indexing

An index is a data structure that provides fast access to records in a file based on the values of one or more attributes (the search key). Without an index, finding records requires a full sequential scan of the entire file. An index is analogous to a book's index — it gives you the page number without reading the whole book.

```
Index Entry: <search_key_value, pointer_to_record>
  pointer_to_record = (block_id, offset_within_block)
```

1. Primary Index (Clustering Index on Ordered File)

A primary index is built on the ordering key field of an ordered (sequentially organized) file. The index entries are ordered in the same order as the data file. One index entry per disk block of the data file (sparse index is typical).

- The data file is physically ordered by the index search key
- Index is sparse — one entry per block, pointing to the first record in each block
- Also called a clustered index because records are clustered by the index key

```
Data file ordered by EmpID:
Block 1: [E001, E002, E003, E004]
Block 2: [E005, E006, E007, E008]
Block 3: [E009, E010, E011, E012]

Primary Index (sparse — one entry per block):
  E001 → Block 1
  E005 → Block 2
  E009 → Block 3
```

2. Secondary Index

A secondary index is built on a non-ordering attribute of the data file. Since the data is NOT ordered by this attribute, each search key value may point to many records scattered across different blocks. Secondary indices are always dense (one entry per record or per distinct key value).

- Data file is NOT ordered by the secondary index key
- Index is dense — one entry per record (or per distinct key value with a bucket of pointers)
- Slower than primary index for range queries due to scattered record locations
- Multiple secondary indices can exist on the same file

```
Data file ordered by EmpID. Secondary index on salary:
Salary 30000 → [Block 3, offset 2]
Salary 45000 → [Block 1, offset 1], [Block 5, offset 3]
Salary 55000 → [Block 2, offset 0]
  (records with same salary are in scattered blocks)
```

3. Clustering Index

A clustering index is built on a non-key ordering attribute of the data file. Records with the same value of the clustering attribute are stored together (clustered) on disk. There is one index entry per distinct value of the clustering attribute.

- File is ordered by the clustering attribute (which is NOT unique)
- One index entry per distinct value of the clustering key
- More records per index entry than primary index (key is non-unique)

```
Data file ordered by DeptNo (non-unique):
Block 1: [E001/D01, E002/D01, E003/D01]
Block 2: [E004/D02, E005/D02, E006/D02]

Clustering Index:
  D01 → Block 1  (first block with D01 records)
  D02 → Block 2
```

Dense Index vs Sparse Index

Aspect	Dense Index	Sparse Index
Definition	One index entry for EVERY record in the data file	One index entry for EVERY BLOCK (or every k-th record)
Index size	Large — same number of entries as records	Small — much fewer entries than records
Access speed	Faster — can locate exact record from index alone	Slightly slower — may need to scan within block
Space cost	High	Low
Applicable to	Any file organization (ordered or unordered)	Only ordered (sequentially organized) files
Example	Secondary index on salary — one entry per employee	Primary index on EmpID — one entry per block
Can answer queries without accessing data file?	Yes (for key fields with projection) — covering index	No — must access data file after index lookup

Multilevel Index

When a single-level index itself becomes too large to fit in memory, a multilevel index (index on index) is used. The outer index is a sparse index on the inner index, and so on recursively. A B+ tree is the standard implementation of a multilevel index.

```
Single-level index: index file with many blocks
  → binary search still requires log2(index_blocks) reads

Two-level (multilevel) index:
  Outer index (sparse, in memory) → Inner index (on disk) → Data file
  Access: 1 lookup in outer + 1 read of inner block + 1 data block read
  = O(log n) with small constant

B+ tree = dynamic multilevel index that stays balanced automatically
```

Q: What is hashing? Explain static hashing. What are the overflow handling / collision resolution techniques?

Asked: 2081 Chaitra; 2079 Chaitra; 2079 Ashwin; 2078 Baishakh; 2075; 2073; 2072; 2071; 2070 (all years) | Marks: [4]

Answer

Hashing

Hashing is a technique to map search key values directly to disk block (bucket) addresses using a hash function h. Given a search key value K, h(K) gives the address of the bucket where records with key K are stored. Hashing provides O(1) average-case access for equality queries.

```
Hash function: h(K) = K mod n   (where n = number of buckets)
Example: h(K) = K mod 7
  K=14 → h(14)=0  K=7 → h(7)=0  K=15 → h(15)=1
  K=3  → h(3)=3   K=11→ h(11)=4  K=23→ h(23)=2
```

Static Hashing

In static hashing, the number of buckets n is fixed at the time the hash file is created. All records are distributed across these n fixed buckets. The hash function maps search keys to bucket numbers in the range [0, n-1].

Structure:

- Hash table: array of n bucket headers
- Each bucket: one or more disk blocks that can hold multiple records
- Bucket capacity: typically b records per bucket
- Overflow bucket: an extra block chained to a full bucket

```
Static hash table with n=5 buckets, h(K) = K mod 5:

Bucket 0: [5, 10, 15, 20] → overflow: [25, 30] (chained)
Bucket 1: [1, 6, 11]
```

```
Bucket 2: [2, 7, 12]
Bucket 3: [3, 8, 13]
Bucket 4: [4, 9, 14]
```

Overflow Handling (Collision Resolution) Techniques

A collision occurs when two different keys hash to the same bucket and the bucket is full. Overflow handling techniques deal with this situation:

Technique 1: Overflow Chaining (Closed Hashing / Separate Chaining)

When a bucket is full, allocate a new overflow bucket and chain it to the original bucket using a pointer. The overflow bucket holds the excess records.

```
Bucket 0: [5, 10, 15] → overflow bucket: [20, 25] → overflow bucket: [30]

Search: hash(K) → bucket 0 → scan → follow overflow chain if not found
Cost:  best case O(1), worst case O(overflow chain length)
```

- Advantages: Simple; no wasted space in other buckets; original hash function unchanged
- Disadvantages: Overflow chains can grow long, degrading performance to O(n)

Technique 2: Open Addressing (Probing)

When a bucket is full, find another empty bucket using a probing sequence. No separate overflow buckets — all records stay in the main hash table.

- Linear probing: try bucket (h(K)+1) mod n, then (h(K)+2) mod n, etc.
- Quadratic probing: try $h(K)+1^2$, $h(K)+2^2$, $h(K)+3^2$, etc.
- Double hashing: use a second hash function h2(K) for probe step

```
Linear probing example (n=5, h(K)=K mod 5):
Insert 5: h(5)=0 → Bucket 0 is free → store in Bucket 0
Insert 10: h(10)=0 → Bucket 0 FULL → try Bucket 1 → store in Bucket 1
Insert 15: h(15)=0 → Bucket 0 FULL → Bucket 1 FULL → Bucket 2 → store
```

Technique 3: Multiple Hashing

Apply a second hash function h2(K) when the first bucket is full. If h2 bucket is also full, apply h3, etc.

Problems with Static Hashing

- Fixed number of buckets: Cannot adapt as data volume grows or shrinks
- Overflow problem: Heavy overflow chains degrade performance to O(n)
- Periodic reorganization: Must periodically rehash entire file when too full — expensive
- Wasted space: Under-utilized buckets waste disk space
- Choosing n: Hard to pick the right n in advance — too small causes overflow; too large wastes space

These problems led to the development of dynamic hashing (extendible hashing) which adapts the number of buckets as data grows.

Q: What is extendible (dynamic) hashing? Explain its structure (directory, global depth, local depth, buckets). What happens on bucket overflow?

Asked: 2081 Chaitra; 2079 Chaitra; 2079 Jestha; 2077; 2076; 2075; 2074; 2073 Magh; 2072 Magh; 2071; 2070 Magh | Marks: [4–6]

Answer

Extendible Hashing (Dynamic Hashing)

Extendible hashing is a dynamic hash structure that can grow and shrink gracefully as the number of records changes. Unlike static hashing, the number of buckets is not fixed — buckets are split (or merged) on demand, avoiding overflow chains and periodic full reorganization.

Key Concepts

Hash Function and Bit Prefixes

A hash function maps each key K to a binary bit string h(K). The directory uses only the first i bits (global depth i) of h(K) to index into the directory.

```
Example hash function: h(K) = K mod 8  →  3-bit binary representation
  h(2)=2=010   h(3)=3=011   h(5)=5=101   h(7)=7=111
  h(24)=0=000  h(19)=3=011  h(12)=4=100  h(26)=2=010

The directory uses the FIRST i bits (most significant bits) as the index.
```

Global Depth (i)

The global depth i is the number of bits of the hash value currently used to index the directory. The directory has 2^i entries. When any bucket overflows and splitting cannot resolve it within the current depth, the global depth is doubled (directory doubled).

Local Depth (j)

Each bucket has a local depth j that indicates how many bits of the hash value were used to create that bucket. j ≤ i always. If j < i, the bucket is pointed to by multiple directory entries (2^(i-j) entries point to the same bucket).

Bucket

A bucket holds a fixed number of records (say, bucket capacity = 2 or 3 records). When a bucket is full and a new record must be inserted, the bucket splits. The local depth of the split bucket increases by 1.

Splitting Rules

```
When bucket B (local depth j) overflows:

Case 1: j < i  (local depth < global depth)
  → Split bucket B into B1 and B2
  → Increase local depth: j = j+1 for both B1 and B2
  → Redistribute records based on the new bit
```

```
  → Update directory: some entries pointing to B now point to B1 or B2
  → Directory size stays the SAME (no doubling needed)

Case 2: j = i  (local depth = global depth)
  → Must double the directory first: i = i+1, size = 2^(i+1)
  → Each old entry is now represented by two new entries (pointing to same
bucket)
  → Then split the overflowing bucket as in Case 1
  → Redistribute records
```

Search and Insert Operations

```
Search(K):
  1. Compute h(K)
  2. Take first i bits → directory index p
  3. Follow pointer: directory[p] → bucket
  4. Scan bucket for K
  Cost: O(1) — 1 directory read + 1 bucket read

Insert(K, record):
  1. Compute h(K); find directory entry; find bucket B
  2. If bucket B has free space: insert directly
  3. If bucket B is FULL: split B (possibly doubling directory first)
     Redistribute all records of B into B1 and B2
     Retry insertion
```

Q: Trace the insertion of keys 2, 3, 5, 7, 24, 19, 12, 26 into an extendible hash table with initial global depth 1 and bucket capacity 2.

Asked: 2081 Chaitra (Regular); 2079 Chaitra (Regular) — most common exam question | Marks: [6]

Answer — Step-by-Step Trace

Setup:

Bucket capacity = 2 records. Initial global depth i = 1 (directory has 2^1 = 2 entries). Hash function: h(K) = K mod 8, use last i bits of binary representation.

```
Hash values (binary, 3 bits):
  h(2)=2=010   h(3)=3=011   h(5)=5=101   h(7)=7=111
  h(24)=0=000  h(19)=3=011  h(12)=4=100  h(26)=2=010

We use LAST i bits (LSB) as directory index — standard convention:
  depth 1: use last 1 bit (0 or 1)
  depth 2: use last 2 bits (00, 01, 10, 11)
```

Initial State (depth=1, 2 buckets):

```
Global depth i=1

Directory:          Buckets:
```

```
  [0] ──────────────▶ Bucket A (local depth j=1): [ ]
  [1] ──────────────▶ Bucket B (local depth j=1): [ ]
```

Insert 2: h(2)=010, last 1 bit = 0 → Directory[0] → Bucket A

```
  [0] ──▶ Bucket A (j=1): [2]
  [1] ──▶ Bucket B (j=1): [ ]
```

Insert 3: h(3)=011, last 1 bit = 1 → Directory[1] → Bucket B

```
  [0] ──▶ Bucket A (j=1): [2]
  [1] ──▶ Bucket B (j=1): [3]
```

Insert 5: h(5)=101, last 1 bit = 1 → Directory[1] → Bucket B → [3,5]

```
  [0] ──▶ Bucket A (j=1): [2]
  [1] ──▶ Bucket B (j=1): [3, 5]
```

Insert 7: h(7)=111, last 1 bit = 1 → Bucket B is FULL (capacity=2)! SPLIT

Bucket B has j=1 = i=1, so we must DOUBLE the directory first (i becomes 2):

```
After doubling (i=2, directory has 4 entries):
  [00] ──▶ Bucket A (j=1): [2]       ← both 00 and 10 point to A since j_A=1
  [01] ──▶ Bucket B (j=1): [3, 5]    ← both 01 and 11 point to B since j_B=1
  [10] ──▶ Bucket A (j=1): [2]
  [11] ──▶ Bucket B (j=1): [3, 5]

Now split Bucket B (j becomes 2):
  Last 2 bits of 3 (011) = 01 → B1
  Last 2 bits of 5 (101) = 01 → B1  (both go to B1!)
  Create B2 for last 2 bits = 11

  Bucket B1 (j=2): [3, 5]  ← entries [01]
  Bucket B2 (j=2): [ ]     ← entries [11]

Insert 7: h(7)=111, last 2 bits = 11 → Bucket B2 → [7]

  [00] ──▶ Bucket A (j=1): [2]
  [01] ──▶ Bucket B1(j=2): [3, 5]
  [10] ──▶ Bucket A (j=1): [2]
  [11] ──▶ Bucket B2(j=2): [7]
```

Insert 24: h(24)=24 mod 8=0=000, last 2 bits = 00 → Bucket A → [2,24]

```
  [00] ──▶ Bucket A (j=1): [2, 24]
  [01] ──▶ Bucket B1(j=2): [3, 5]
  [10] ──▶ Bucket A (j=1): [2, 24]
  [11] ──▶ Bucket B2(j=2): [7]
```

Insert 19: h(19)=19 mod 8=3=011, last 2 bits = 11 → Bucket B2 → [7,19]

```
  [00] ──▶ Bucket A (j=1): [2, 24]
  [01] ──▶ Bucket B1(j=2): [3, 5]
```

```
  [10] ──► Bucket A (j=1): [2, 24]
  [11] ──► Bucket B2(j=2): [7, 19]
```

Insert 12: h(12)=12 mod 8=4=100, last 2 bits = 00 → Bucket A → FULL! SPLIT

Bucket A has j=1 < i=2, so NO directory doubling needed (Case 1):

```
Split Bucket A (j becomes 2):
  Last 2 bits of 2  (010) = 10 → A2
  Last 2 bits of 24 (000) = 00 → A1
  Last 2 bits of 12 (100) = 00 → A1

  Bucket A1 (j=2): [24, 12]  ← entries [00]
  Bucket A2 (j=2): [2]       ← entries [10]

Update directory:
  [00] ──► Bucket A1(j=2): [24, 12]
  [01] ──► Bucket B1(j=2): [3, 5]
  [10] ──► Bucket A2(j=2): [2]
  [11] ──► Bucket B2(j=2): [7, 19]
```

Insert 26: h(26)=26 mod 8=2=010, last 2 bits = 10 → Bucket A2 → [2,26]

```
FINAL STATE (i=2):

Directory:                      Buckets:
  [00] ──────────────────────► A1 (j=2): [24, 12]
  [01] ──────────────────────► B1 (j=2): [3, 5]
  [10] ──────────────────────► A2 (j=2): [2, 26]
  [11] ──────────────────────► B2 (j=2): [7, 19]

Global depth i=2, all local depths j=2.
All 8 keys inserted: 2,3,5,7,24,19,12,26 ✓
```

Q: What is a B+ tree? Explain its structure, properties, and order definition. How does it differ from a B-tree?

Asked: ALL years 2070–2082 — asked every exam | Marks: [4–6]

Answer

B+ Tree

A B+ tree is a self-balancing, multi-level index structure where all search key values and pointers to data records appear in the leaf nodes. Internal nodes contain only search keys and pointers to child nodes — they do NOT contain pointers to data records. All leaf nodes are linked together as a doubly-linked list, enabling efficient range scans.

Order Definition

IMPORTANT: 'Order n' (or 'order p') has two common conventions:

Convention 1 (Silberschatz textbook): order n means each node can have at most n search keys and n+1 pointers.
Convention 2: order n means maximum n keys per leaf node (sometimes called 'degree n' or 'order 2n').
TU-IOE exam questions typically use: ORDER n = max n keys per internal node = max n keys per leaf.
In this document, ORDER n means: max n keys, min □n/2□ keys (except root).

B+ Tree Properties

```
For a B+ tree of ORDER n (max n keys per node):

Internal (non-leaf) nodes:
  • Max keys: n
  • Max pointers: n+1  (one more than keys)
  • Min keys: □n/2□ - 1  (except root: min 1 key)
  • Min pointers: □n/2□  (except root: min 2)
  • Keys are search keys only (no record pointers)

Leaf nodes:
  • Max keys: n  (each key has a pointer to the data record)
  • Min keys: □n/2□
  • One pointer to the next leaf (for range scan linked list)
  • Each entry: <key, pointer to record>

General properties:
  • All leaves are at the SAME level (perfectly balanced height)
  • Height h = O(log_□n/2□(N)) where N = number of records
  • Search: traverse from root to leaf — exactly h node accesses
  • Every path from root to leaf has the same length
```

B+ Tree vs B-Tree

Aspect	B-Tree (B-tree)	B+ Tree
Record pointers	In ALL nodes (internal + leaf)	ONLY in leaf nodes
Internal node keys	Key + pointer to record	Key only (no record pointer)
Leaf node linking	NOT linked	All leaves linked as a list
Range queries	Requires tree traversal (expensive)	Efficient — scan leaf chain
Space in internal nodes	Less — each key takes a record pointer	More keys per internal node — wider fan-out
Common use	Older systems; file systems	Database indices (standard choice)

B+ Tree Node Structure

```
Internal node (order n=3, max 3 keys, 4 pointers):

┌────┬────┬────┬────┬────┬────┬────┐
│ P0 │ K1 │ P1 │ K2 │ P2 │ K3 │ P3 │
└────┴────┴────┴────┴────┴────┴────┘

  P0 points to subtree with keys < K1
  P1 points to subtree with K1 ≤ keys < K2
```

```
  P2 points to subtree with K2 ≤ keys < K3
  P3 points to subtree with keys ≥ K3

Leaf node (order n=3, max 3 keys):
┌────┬──────┬────┬──────┬────┬──────┬──────────┐
│ K1 │ ptr1 │ K2 │ ptr2 │ K3 │ ptr3 │ next_leaf│
└────┴──────┴────┴──────┴────┴──────┴──────────┘
  ptr1 → record with key K1 (in data file)
  next_leaf → pointer to next leaf node (for range scan)
```

B+ Tree Insertion Algorithm

```
Insert(key K, record pointer ptr):
  1. Search for leaf node L where K belongs (root-to-leaf traversal)
  2. If L has free space (keys < n):
       Insert (K, ptr) into L in sorted order. DONE.
  3. If L is FULL (keys = n):
       SPLIT L into L and L_new:
         - Put first □(n+1)/2□ keys in L
         - Put remaining keys in L_new
         - Copy (NOT move) the middle key K_mid up to parent
           (Note: in B+ tree, middle key stays in L_new leaf AND is pushed up)
       Insert pointer to L_new in parent of L
       If parent is also full: split parent (push up — no copy for internal nodes)
       Splits propagate upward until a non-full node is found
       If root splits: create new root with one key and two children

Key rule for B+ tree splits:
  - Leaf split: COPY middle key to parent (key stays in leaf too)
  - Internal node split: PUSH UP middle key (key moves to parent, not copied)
```

Q: Insert the following keys into an empty B+ tree of order 3: 10, 20, 30, 40, 50. Show the tree after each insertion.

Asked: 2082 Kartik; 2074 Bhadra (also: 10,20,30,25,35) | Marks: [4–6]

Answer — B+ Tree Insertions (Order 3)

Order 3 means: max 3 keys per node, min □3/2□ = 2 keys (except root: min 1).
Max 4 pointers per internal node, max 3 search-key/record-pointer pairs per leaf.
When a node has 3+1=4 keys (overflow), SPLIT: first 2 keys stay, last 2 in new node, middle key (3rd key, index 2 from 0) is pushed/copied up.

Step 1: Insert 10

```
Leaf: [10]

No split needed.
```

Step 2: Insert 20

```
Leaf: [10 | 20]

No split needed (2 keys ≤ max 3).
```

Step 3: Insert 30

```
Leaf: [10 | 20 | 30]

3 keys = max 3. Full, but no overflow yet (we overflow at 4 keys).
```

Step 4: Insert 40 → Leaf OVERFLOWS (4 keys: 10,20,30,40)

Leaf has 4 keys — overflow! Split the leaf:

```
Split leaf [10,20,30,40]:
  Left leaf  L1: [10, 20]         (first ⌊4/2⌋ = 2 keys)
  Right leaf L2: [30, 40]         (remaining 2 keys)
  Copy up: 30 (first key of L2) → new root

Tree after insert 40:
           [30]             ← Root (internal node)
          /    \
      [10,20]  [30,40]      ← Leaf nodes, linked: L1 → L2
```

Step 5: Insert 50 → goes into right leaf [30,40]

```
h(50): 50 > 30 → right leaf L2
L2 = [30, 40, 50] — 3 keys, no overflow.

Tree after insert 50:
              [30]
             /    \
        [10,20]  [30,40,50]     ← L2 now has 3 keys (max)
```

Additional: Insert 25 → goes into L1 [10,20] ... wait, 25>20 and 25<30 → left leaf

```
Insert 25: 25 < 30 → left leaf L1 = [10, 20]
L1 = [10, 20, 25] — 3 keys, no overflow.

              [30]
             /    \
      [10,20,25]  [30,40,50]
```

Additional: Insert 35 → right leaf [30,40,50] + 35 = OVERFLOW (4 keys: 30,35,40,50)

```
Right leaf L2 overflows: [30,35,40,50]
Split L2:
  L2a: [30, 35]
  L2b: [40, 50]
  Copy up: 40 (first key of L2b) → parent root

Root was [30], now gets 40 inserted: [30, 40]  (2 keys ≤ max 3, no root split)

FINAL TREE after 10,20,30,40,50,25,35:
```

```
                [30 | 40]
              /     |     \
        [10,20,25] [30,35] [40,50]
          L1   ──►   L2a ──► L2b    (leaf chain)
```

Q: Insert keys 5, 8, 1, 7, 3, 12, 9, 6 into an empty B+ tree of order 3. Show tree state after all insertions.
Asked: 2081 Ashwin (Back); 2072 Ashwin (Regular) | Marks: [4–6]

Answer — B+ Tree Insertions: 5, 8, 1, 7, 3, 12, 9, 6 (Order 3)

Insert 5, 8, 1:

```
After 5:      [5]
After 8:      [5 | 8]
After 1:      [1 | 5 | 8]   ← sorted insertion
```

Insert 7 → Leaf OVERFLOWS: [1,5,7,8] (4 keys)

```
Split [1,5,7,8]:
  L1: [1, 5]   L2: [7, 8]   Copy up: 7


       [7]
      /   \
   [1,5]  [7,8]
```

Insert 3 → 3 < 7 → L1 = [1,5] → [1,3,5]

```
       [7]
      /   \
  [1,3,5]  [7,8]
```

Insert 12 → 12 > 7 → L2 = [7,8] → [7,8,12]

```
       [7]
      /   \
  [1,3,5]  [7,8,12]
```

Insert 9 → 9 > 7 → L2 = [7,8,9,12] OVERFLOW

```
Split [7,8,9,12]:
  L2a: [7,8]   L2b: [9,12]   Copy up: 9
Root [7] gets 9 → [7,9]  (2 keys, no overflow)


          [7 | 9]
         /   |   \
    [1,3,5] [7,8] [9,12]
```

Insert 6 → 6 > root.keys[0]=7? No, 6<7 → L1 = [1,3,5] → [1,3,5,6] OVERFLOW

```
Split [1,3,5,6]:
  L0: [1,3]   L1: [5,6]   Copy up: 5
Root [7,9] gets 5 → [5,7,9]  (3 keys = max, no overflow)


             [5 | 7 | 9]
            /   |   |   \
        [1,3] [5,6] [7,8] [9,12]
  L0 ──▶ L1 ──▶ L2a ──▶ L2b   (leaf chain)
```

Final B+ tree after inserting 5,8,1,7,3,12,9,6 (order 3):

```
Root:       [5 | 7 | 9]
           /   |   |   \
Leaves: [1,3] [5,6] [7,8] [9,12]
          └──▶──▶──▶┘   (linked list)
```

Q: Insert 10, 20, 30, 25, 35 into a B+ tree of order 3. Then delete keys 20 and 30. Show the tree at each step.
Asked: 2080 Ashwin (Back Exam) | Marks: [6]

Answer — B+ Tree Insert then Delete (Order 3)

PART 1: Insertions — 10, 20, 30, 25, 35

Insert 10, 20, 30:

```
After 10,20,30:  Leaf [10 | 20 | 30]  (3 keys = max 3, still OK)
```

Insert 25 → sorted: [10,20,25,30] OVERFLOW

```
Split [10,20,25,30]:
  L1: [10,20]   L2: [25,30]   Copy up: 25


       [25]
      /    \
  [10,20] [25,30]
```

Insert 35 → 35 > 25 → L2 = [25,30,35]

```
       [25]
      /    \
  [10,20] [25,30,35]

Tree after all insertions: 10,20,30,25,35
```

PART 2: Deletion Algorithm

```
Delete(key K):
  1. Find leaf L containing K
  2. Remove K from L
  3. If L has ≥ ⌊n/2⌋ keys after deletion: DONE
  4. If L has < ⌊n/2⌋ keys (underflow):
     a. Try to BORROW from a sibling leaf (redistribute):
           If left sibling has extra: borrow its rightmost key
           If right sibling has extra: borrow its leftmost key
           Update the parent separator key accordingly
     b. If no sibling has extra keys: MERGE with a sibling:
           Combine L and sibling into one leaf
           Remove the separator key from parent
           If parent underflows: recurse upward
           For internal node merges: PULL DOWN the separator (not copy)
```

Delete 20 from leaf L1 [10,20]:

```
Before: Root[25]; L1=[10,20]; L2=[25,30,35]

Remove 20 from L1 → L1 = [10]
L1 has 1 key, min = ⌊3/2⌋ = 2 keys → UNDERFLOW!

Check right sibling L2 = [25,30,35] — has 3 keys > min (can lend):
  Borrow leftmost key of L2: 25 → move to L1
  L1 = [10, 25]  L2 = [30, 35]
  Update parent separator: was 25, now = 30 (new first key of L2)

Tree after deleting 20:
       [30]
      /    \
  [10,25] [30,35]
```

Delete 30 from leaf L2 [30,35]:

```
Before: Root[30]; L1=[10,25]; L2=[30,35]

Remove 30 from L2 → L2 = [35]
L2 has 1 key, min = 2 → UNDERFLOW!

Check left sibling L1 = [10,25] — has 2 keys = min (cannot lend).
No right sibling (L2 is rightmost).
→ MERGE L1 and L2:
  Combined leaf: [10, 25, 35]
  Pull down separator 30 from parent (parent loses it)
  Root[30] loses its only key → Root becomes empty!
  → L_merged becomes the new root leaf

FINAL TREE after deleting 20 and 30:

  [10 | 25 | 35]   ← single leaf (tree height reduced to 1)
```

Q: Insert keys 10, 20, 30, 40, 50, 60, 70 into an empty B+ tree of order 3. Show the final tree.

Asked: 2078 Chaitra (Regular); 2073 Magh (Back) — also: 10,30,50,20,40,60,70 | Marks: [4–6]

Answer — B+ Tree Insertions: 10, 20, 30, 40, 50, 60, 70 (Order 3)

Insert 10, 20, 30 → [10,20,30]

```
[10 | 20 | 30]  (3 keys = max, no overflow yet)
```

Insert 40 → [10,20,30,40] OVERFLOW → Split

```
Split [10,20,30,40]: L1=[10,20], L2=[30,40], copy up 30
       [30]
      /    \
  [10,20] [30,40]
```

Insert 50 → right leaf [30,40,50]

```
       [30]
      /    \
  [10,20] [30,40,50]
```

Insert 60 → right leaf [30,40,50,60] OVERFLOW → Split

```
Split [30,40,50,60]: L2=[30,40], L3=[50,60], copy up 50
Root [30] gets 50 → [30,50]  (2 keys, OK)


          [30 | 50]
         /    |    \
    [10,20] [30,40] [50,60]
```

Insert 70 → rightmost leaf [50,60,70]

```
          [30 | 50]
         /    |    \
    [10,20] [30,40] [50,60,70]
```

FINAL tree after inserting 10,20,30,40,50,60,70 (order 3):

```
Internal root:   [30 | 50]
                /    |    \
Leaves:   [10,20]  [30,40]  [50,60,70]
             └──────►──────►┘   (linked)
```

Q: Construct a B+ tree of order 3 by inserting: 1, 4, 7, 10, 17, 21, 31, 25, 19, 20, 28.
Asked: 2072 Magh (Back Exam) | Marks: [4]

Answer — B+ Tree: 1, 4, 7, 10, 17, 21, 31, 25, 19, 20, 28 (Order 3)

Insert 1, 4, 7:

```
[1 | 4 | 7]
```

Insert 10 → [1,4,7,10] OVERFLOW

```
Split: L1=[1,4]  L2=[7,10]  copy up 7
     [7]
    /   \
 [1,4] [7,10]
```

Insert 17 → 17>7 → L2=[7,10,17]

```
     [7]
    /   \
 [1,4] [7,10,17]
```

Insert 21 → L2=[7,10,17,21] OVERFLOW

```
Split: L2=[7,10]  L3=[17,21]  copy up 17
Root [7] → [7,17]

        [7 | 17]
       /   |    \
   [1,4] [7,10] [17,21]
```

Insert 31 → 31>17 → L3=[17,21,31]

```
        [7 | 17]
       /   |    \
   [1,4] [7,10] [17,21,31]
```

Insert 25 → L3=[17,21,25,31] OVERFLOW

```
Split: L3=[17,21]  L4=[25,31]  copy up 25
Root [7,17] → [7,17,25]  (3 keys = max, OK)

          [7 | 17 | 25]
         /    |    |    \
     [1,4] [7,10] [17,21] [25,31]
```

Insert 19 → 17≤19<25 → L3=[17,21] → [17,19,21]

```
          [7 | 17 | 25]
         /    |    |    \
     [1,4] [7,10] [17,19,21] [25,31]
```

Insert 20 → L3=[17,19,20,21] OVERFLOW

```
Split: L3a=[17,19]  L3b=[20,21]  copy up 20
```

```
Root [7,17,25] gets 20 → [7,17,20,25] OVERFLOW! Root splits.

Split root [7,17,20,25]:
  Left internal:  [7,17]    (first 2 keys)
  Right internal: [25]      (last key)
  Push up: 20 → new root

  children of left  [7,17]: L1[1,4], L2[7,10], L3a[17,19]
  children of right [25]:   L3b[20,21], L4[25,31]

                  [20]
               /       \
           [7|17]        [25]
          / |   \       /    \
      [1,4][7,10][17,19] [20,21][25,31]
```

Insert 28 → 28>20 → right subtree [25] → 25≤28 → L4=[25,31] → [25,28,31]

```
FINAL B+ tree after 1,4,7,10,17,21,31,25,19,20,28:

                       [20]
                     /       \
              [7 | 17]      [25]
             /   |   \     /    \
         [1,4] [7,10] [17,19] [20,21] [25,28,31]
    L1 ──▶ L2 ──▶ L3 ──▶ L4 ──▶ L5   (leaf chain)
```

SUMMARY TABLE — Chapter 6 Question Coverage

All Chapter 6 questions from TU-IOE BCT examination papers (2070–2082) and their coverage in this document.

No.	Question / Topic	Asked In	Marks	Status
C1	File Organization — heap, sequential, hashed, B+ tree-based	All years 2070–2082	[4]	Done
C2	Ordered Indices — primary, secondary, clustering index types	2081 Ashwin, 2080, 2078 Poush, 2074, 2073 Magh, 2072 Ashwin, 2071, 2070 Magh	[4]	Done
C3	Dense vs Sparse Index — definitions, examples, when to use each	2081 Ashwin, 2079 Ashwin, 2078 Poush, 2075 Bhadra, 2072 Ashwin, 2071	[4]	Done
C4	Multilevel Index — structure and cost benefit	2078 Poush, 2074, 2073 Magh	[4]	Done
C5	Static Hashing — hash function, bucket overflow, collision handling	2081 Chaitra, 2079 Chaitra, 2079 Ashwin, 2078 Baishakh, 2075, 2073, 2072, 2071, 2070 (all)	[4]	Done
C6	Extendible (Dynamic) Hashing — global/local depth, directory, splitting	2081 Chaitra, 2079 Chaitra, 2079 Jestha, 2077, 2076, 2075, 2074, 2073 Magh, 2072 Magh, 2071	[4–6]	Done
C7	B+ Tree — structure, properties, order definition	All years 2070–2082	[4–6]	Done
T1	B+ Tree Insertion: 10,20,30,40,50 (order 3) — 2082 / 2074	2082 Kartik, 2074 Bhadra	[4–6]	Done
T2	B+ Tree Insertion: 5,8,1,7,3,12,9,6 (order 3) — 2081 Ashwin / 2072	2081 Ashwin, 2072 Ashwin	[4–6]	Done
T3	B+ Tree Insert+Delete: 10,20,30,25,35; delete 20,30 (order 3) — 2080	2080 Ashwin	[6]	Done
T4	B+ Tree Insertion: 10,20,30,40,50,60,70 (order 3) — 2078 Chaitra / 2073	2078 Chaitra, 2073 Magh	[4–6]	Done
T5	B+ Tree Insertion: 1,4,7,10,17,21,31,25,19,20,28 (order 3) — 2072 Magh	2072 Magh	[4]	Done
T6	Extendible Hashing trace: 2,3,5,7,24,19,12,26 (depth 1) — 2081/2079	2081 Chaitra, 2079 Chaitra	[6]	Done
T7	Extendible Hashing trace: 2,3,5,7,24,19 — 2076	2076 Baisakh	[4]	Done

Notation: Order n = max n keys per node; min □n/2□ keys (except root). Leaf splits: COPY middle key up (stays in leaf). Internal splits: PUSH UP middle key (leaves internal node). For extendible hashing: i = global depth, j = local depth; directory has 2^i entries; use last i bits of hash value for indexing.

— End of Chapter 6 Answer Bank —